Theory of Computation

Lecture 14

Computation and Its Limitations!

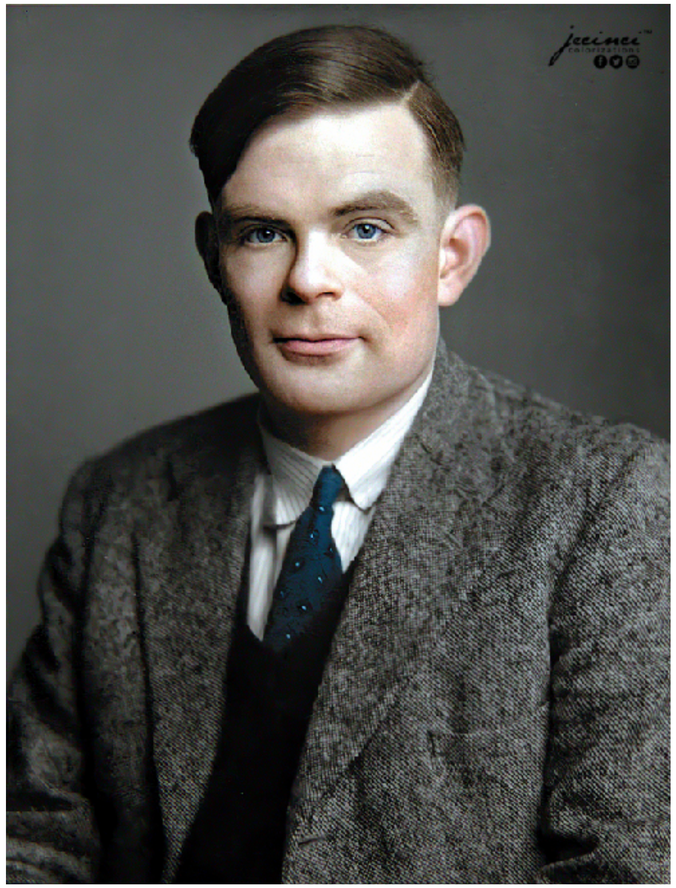
$$A_{\mathrm{TM}} = \{\langle M, w\rangle : M \text{ accepts } w\}$$

$$\mathrm{ALL}_{\mathrm{CFL}} = \{\langle G\rangle : \mathscr{L}(G) = \Sigma^*\}$$

$$\mathrm{HALT}_{\mathrm{TM}} = \{\langle M, w\rangle : M \text{ halts on } w\}$$

$$BB = \{(n, BB(n))\}$$

$$\mathrm{FOL}_{\mathrm{VALID}} = \{\phi \mid \phi \text{ is a valid FOL formula}\}$$



Alan Turing

On Computable Numbers, with an Application to the Entscheidungsproblem

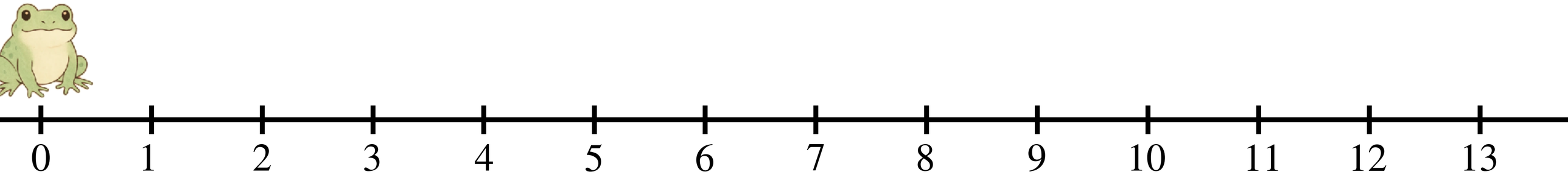
The frog can pick from total Turing Machines that map $\mathbb{N} \to \mathbb{N}$ (computable functions). This set is enumerable.

Let the hunter's strategy be $h(t)$ defined as follows:

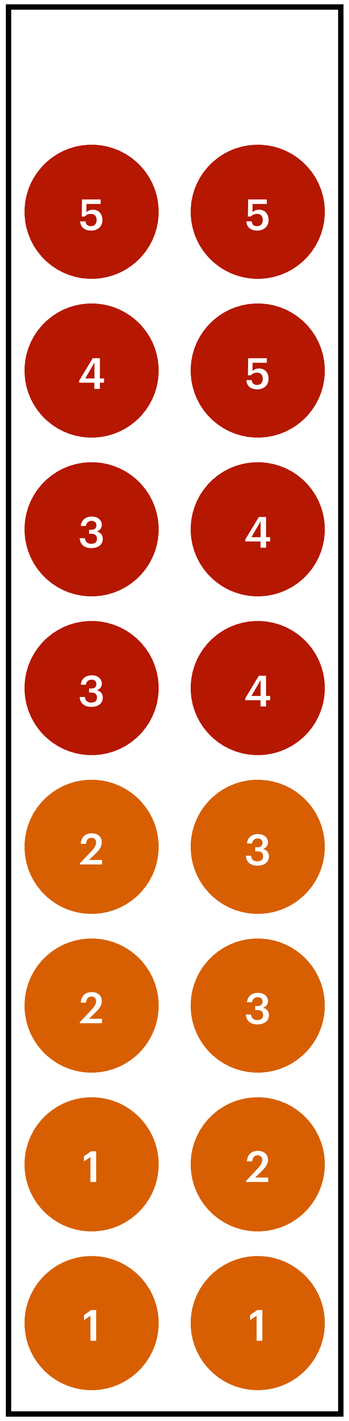
Consider an enumeration $f_1, f_2, \ldots, f_k, \ldots$

Let $h(t) = f_t(t)$.

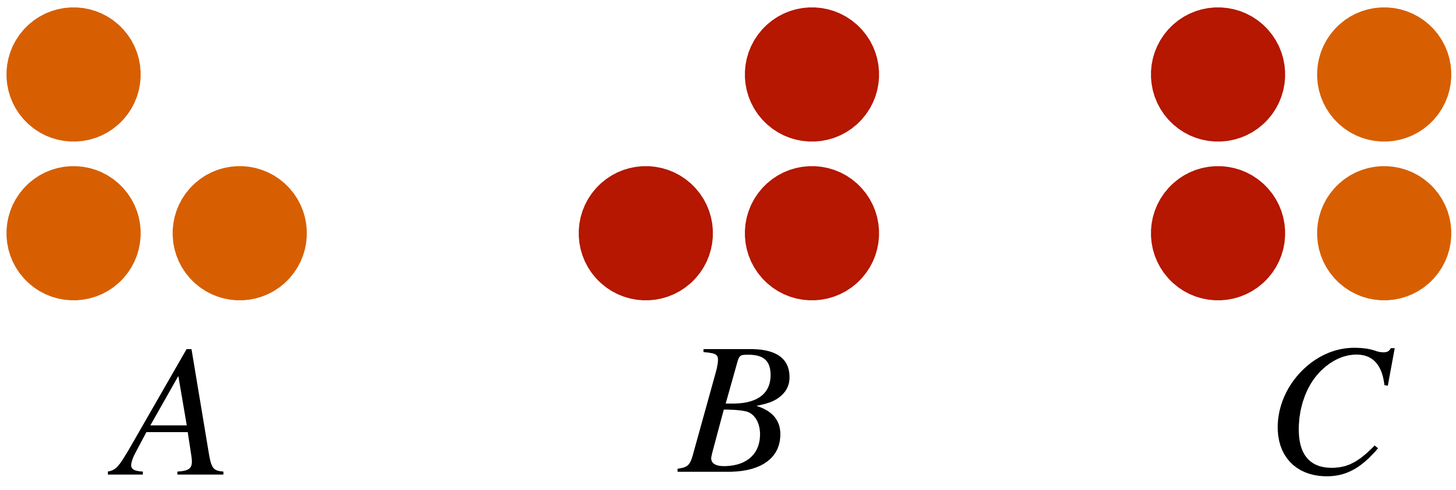
Is $h(t)$ computable?



Post Correspondence Problem



$$AACBB$$



MPCP $\leq_T$ PCP

A TM for PCP can solve MPCP.

i	$f(i)$	$g(i)$
1	1	111
2	10111	10
3	10	0

Suppose I were to solve MPCP on this

i	$f(i)$	$g(i)$
0	$*1*$	$*1*1*1$
1	$1*$	$*1*1*1$
2	$1*0*1*1*1*$	$*1*0$
3	$1*0*$	$*0$
4	$\$$	$*\$$

Then I can just solve PCP on this.

$A_{\mathrm{TM}} \leq_T$ **MPCP**

That is given (M, w), there is an instance (T, f, g) of MPCP such that

$$w \in \mathscr{L}(M) \iff (T, f, g) \text{ has a solution}$$

Each partial solution will represent the prefix of an accepting computation history of M on w of the form $\#C_0\#C_1\#\cdots\#C_k\#\cdots$

The MPCP will have a solution when there is an accepting run.

$A_{\mathrm{TM}} \leq_T$ **MPCP** $\leq_T$ **PCP**

Therefore PCP is undecidable.

The number of tiles depend on the size of the TM.
The number of symbols depend on the tape alphabet.

An Application

Given two CFG G_1 and G_2, it is undecidable if $\mathscr{L}(G_1) \cap \mathscr{L}(G_2) = \emptyset$.

Suppose not.

Given an arbitrary instance of MPCP (T, f, g), consider the grammar G_1:

$$S \to (f(i))^R S g(i) \;\mid\; (f(1))^R \$ g(1) \qquad \forall i \in T$$

Let G_2 be the grammar of palindromes.

Post Correspondence Problem

Given a fixed set of tiles $T = \{1, \ldots, k\}$, and functions $f : T \to \Sigma^*$ and $g : T \to \Sigma^*$, is there a sequence of tiles ($w \in T^+$) such that $\hat{f}(w) = \hat{g}(w)$?

If $|\Sigma| = 2$, PCP is decidable.

Else if $|T| = 1$ it is decidable.

Else if $|T| = 2$ it is decidable.

$|T| = 3$ or $|T| = 4$ is unknown!

Else if $|T| \geq 5$ it is undecidable.

Universality of Turing Machines

$$\mathrm{ALL}_{\mathrm{CFL}} = \{\langle G \rangle : \mathscr{L}(G) = \Sigma^*\}$$

Clearly, $\mathrm{ALL}_{\mathrm{CFL}} \leq_T \mathrm{ALL}_{\mathrm{TM}}$

And hence $\mathrm{ALL}_{\mathrm{TM}}$ is undecidable.

Empty Turing Machines

$$L = \{\langle G_1, G_2 \rangle : \mathscr{L}(G_1) \cap \mathscr{L}(G_2) = \varnothing\}$$

Empty Turing Machines

$$L = \{\langle G_1, G_2 \rangle : \mathscr{L}(G_1) \cap \mathscr{L}(G_2) = \varnothing\}$$

Clearly, $L \leq_T \mathrm{Empty}_{\mathrm{TM}}$

And hence $\mathrm{Empty}_{\mathrm{TM}}$ is undecidable.

Turing Machines with Finite Languages

$$L = \{ \langle M \rangle : |\mathscr{L}(M)| \text{ is finite } \}$$

Which problem should we reduce to this?

Turing Machines with Finite Languages

$$L = \{ \langle M \rangle : |\mathscr{L}(M)| \text{ is finite } \}$$

If w is a solution to PCP, then so is w^k

Which problem should we reduce to this?

Turing Machines with Finite Languages

$$L = \{\langle M \rangle : |\mathscr{L}(M)| \text{ is finite } \}$$

Given an instance of PCP, covert it to two grammars G_1 and G_2 (as in the reduction of intersection non-emptiness of CFGs). Consider the TM M that accepts the language $\mathscr{L}(G_1) \cap \mathscr{L}(G_2)$.

$\mathscr{L}(M)$ is finite $\iff$ PCP has no solution

More Turing Machine Problems!

$$L = \{\langle M \rangle : |\mathscr{L}(M)| = 72\}$$

More Turing Machine Problems!

$$L = \{\langle M \rangle : |\mathscr{L}(M)| = 72\}$$

Consider an instance of M' for $\mathrm{Empty}_{\mathrm{TM}}$

Let # be a symbol not in the alphabet of M'.

Define M such that it has the language $\{\#, \#^2, \ldots, \#^{72}\} \cup \mathscr{L}(M')$.

More Turing Machine Problems!

$$L = \{\langle M \rangle : |\mathscr{L}(M)| = 72\}$$

$$M' \in \text{Empty}_{\text{TM}} \iff M \in L$$

Hence L is not decidable.

Another Proof

$$L = \{\langle M \rangle : |\mathscr{L}(M)| = 72\}$$

Let $S = \{s_1, s_2, \ldots, s_{72}\}$ be a fixed set of 72 strings.
There exists a TM M_S such that $\mathscr{L}\left(M_S\right) = S$.

So the answer is not always a yes and not always a no!

Another Proof

$$L = \{\langle M \rangle : |\mathscr{L}(M)| = 72\}$$

Consider $\langle M, w \rangle$ be an arbitrary instance of the halting problem.

Construct M' such that on input x,

M' simulates M on w.

If M halts on w, check if $x \in S$.

Another Proof

$$L = \{\langle M \rangle : |\mathscr{L}(M)| = 72\}$$

Consider $\langle M, w \rangle$ be an arbitrary instance of the halting problem.

Construct M' such that on input x,

M' simulates M on w.

If M halts on w, check if $x \in S$.

What is the language of M'?

Another Proof

$$L = \{\langle M \rangle : |\mathscr{L}(M)| = 72\}$$

Consider $\langle M, w \rangle$ be an arbitrary instance of the halting problem.

Construct M' such that on input x,

M' simulates M on w.

If M halts on w, check if $x \in S$.

$$\langle M' \rangle \in L \iff \langle M, w \rangle \in \mathrm{HALT}_{\mathrm{TM}}$$

Another Proof

$$L = \{\langle M \rangle : |\mathscr{L}(M)| = 72\}$$

$$\langle M' \rangle \in L \iff \langle M, w \rangle \in \mathrm{HALT}_{\mathrm{TM}}$$

If L was recursive, then so would be $\mathrm{HALT}_{\mathrm{TM}}$.

Another Proof

$$L = \{\langle M \rangle : |\mathscr{L}(M)| = 72\}$$

Is L recursively enumerable? It is co-recursively enumerable?

Neither!

More Turing Machine Problems!

$$L = \{\langle M \rangle : |\mathscr{L}(M)| = 72\}$$

$$L = \{\langle M \rangle : |\mathscr{L}(M)| = 73\}$$

$$L = \{\langle M \rangle : |\mathscr{L}(M)| = 74\}$$

$$L = \{\langle M \rangle : |\mathscr{L}(M)| = 75\}$$

$$L = \{\langle M \rangle : |\mathscr{L}(M)| = 76\}$$

$$\ldots$$

Rice's Theorem

Let P be a non-trivial property of recursively enumerable languages.

$$P : \mathrm{RE}(\Sigma) \rightarrow \{0,1\}$$

Henry Gordon Rice

Rice's Theorem

Let P be a non-trivial property of recursively enumerable languages.

That is, there exists a language L_1 for which it holds and a language L_2 for which it does not hold.

Henry Gordon Rice

Rice's Theorem

Let P be a non-trivial property of recursively enumerable languages.

$$P : \mathrm{RE}(\Sigma) \to \{0,1\}$$

$\exists L_1, L_2 \in \mathrm{RE}(\Sigma)$ such that

$$P(L_1) \neq P(L_2).$$

Henry Gordon Rice

Rice's Theorem

Let P be a non-trivial property of recursively enumerable languages.

Then $L_P = \{\langle M \rangle : P(\mathscr{L}(M)) \text{ holds}\}$ is not recursive.

Henry Gordon Rice

Rice's Theorem

$$\mathrm{HALT}_{\mathrm{TM}} \leq_T \{\langle M \rangle : P(M) \text{ holds}\}$$

Suppose P holds for M_1.

Consider $\langle M, w \rangle$ be an arbitrary instance of the halting problem.

Construct $A_{M,w}$ such that on input x,

$A_{M,w}$ simulates M on w.

If M halts on w, check if $x \in \mathscr{L}(M_1)$.

Rice's Theorem

$$\mathrm{HALT}_{\mathrm{TM}} \leq_T \{\langle M \rangle : P(M) \text{ holds}\}$$

$$\langle M, w \rangle \in \mathrm{HALT}_{\mathrm{TM}} \iff \langle M' \rangle \in \{\langle M \rangle : P(M) \text{ holds}\}$$

And therefore no non-trivial property of a Turing Machine is decidable.

Applications

The following are undecidable:

$$\{\langle M\rangle : \mathscr{L}(M) \text{ is empty}\}$$

$$\{\langle M\rangle : \mathscr{L}(M) \text{ is finite}\}$$

$$\{\langle M\rangle : \mathscr{L}(M) \text{ is regular}\}$$

$$\{\langle M\rangle : \mathscr{L}(M) \text{ is context-free}\}$$

$$\{\langle M\rangle : \mathscr{L}(M) \text{ is recursive}\}$$

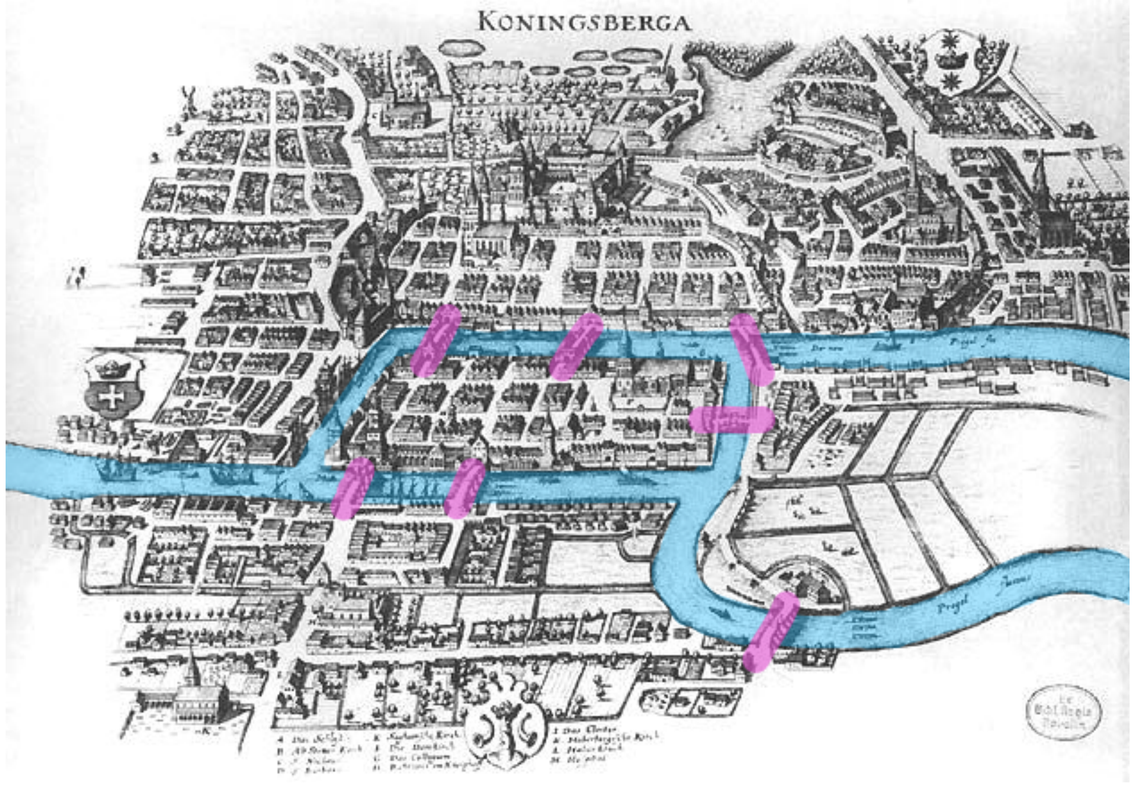
Applications

Checking if a hello-world program indeed prints hello world is undecidable!

Reachability of an LOC of a program is undecidable.

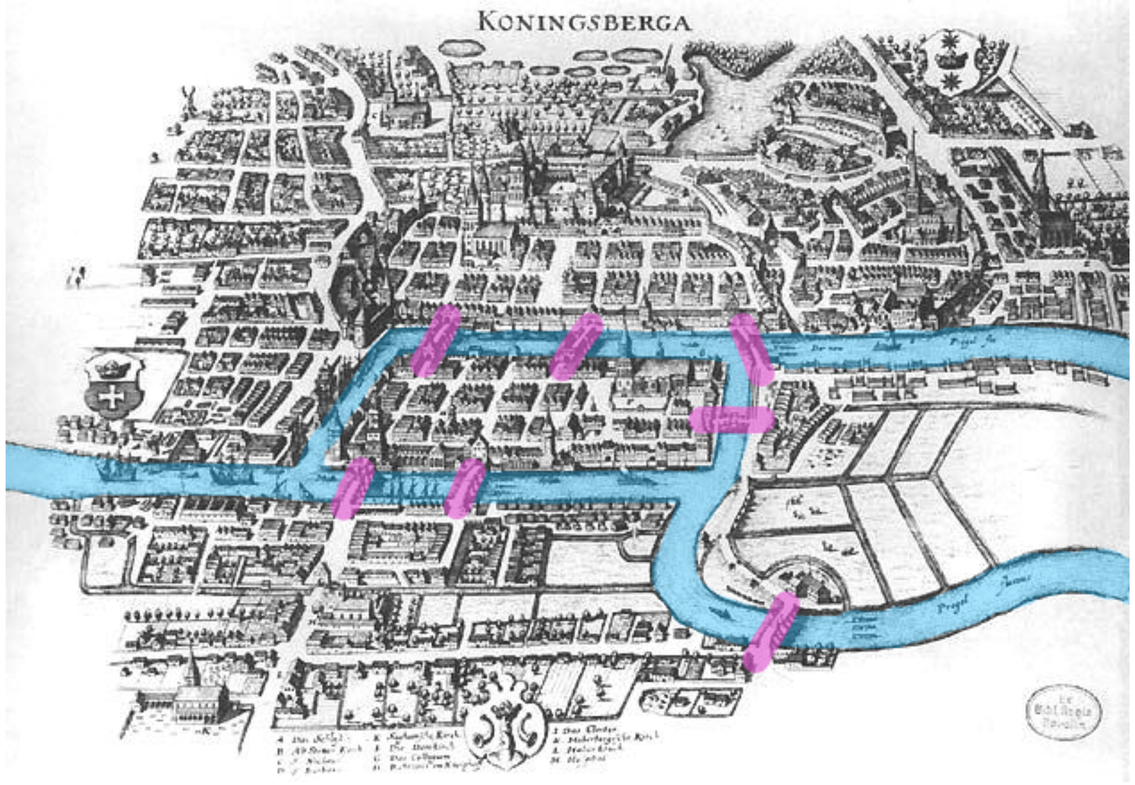
Checking if an invariant holds is undecidable.

What Can Be Computed?



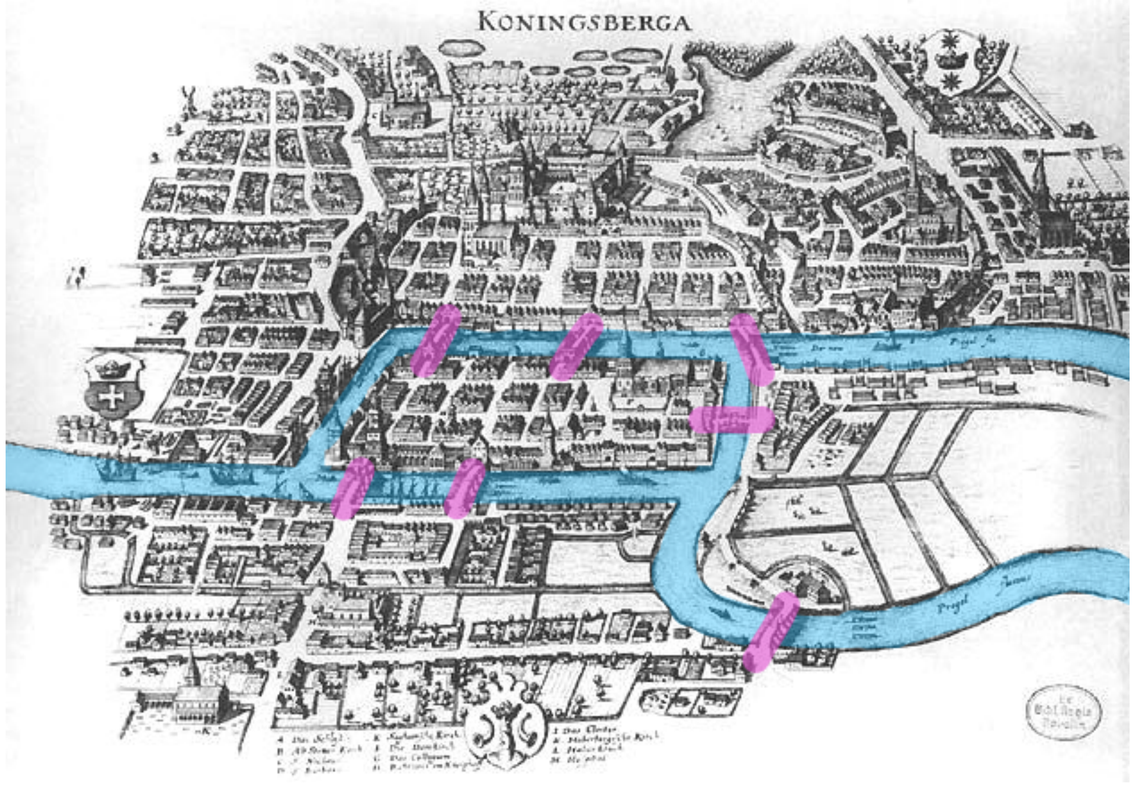
Given an undirected graph, the existence of an Eulerian cycle is decidable in $O(|V| + |E|)$.

Assume G is connected.



How would a Turing Machine compute this?

Given an undirected graph, the existence of an Eulerian cycle is decidable in $O(|V| + |E|)$.



Given an undirected graph, the existence of an Eulerian cycle is decidable in $O(|V| + |E|)$.

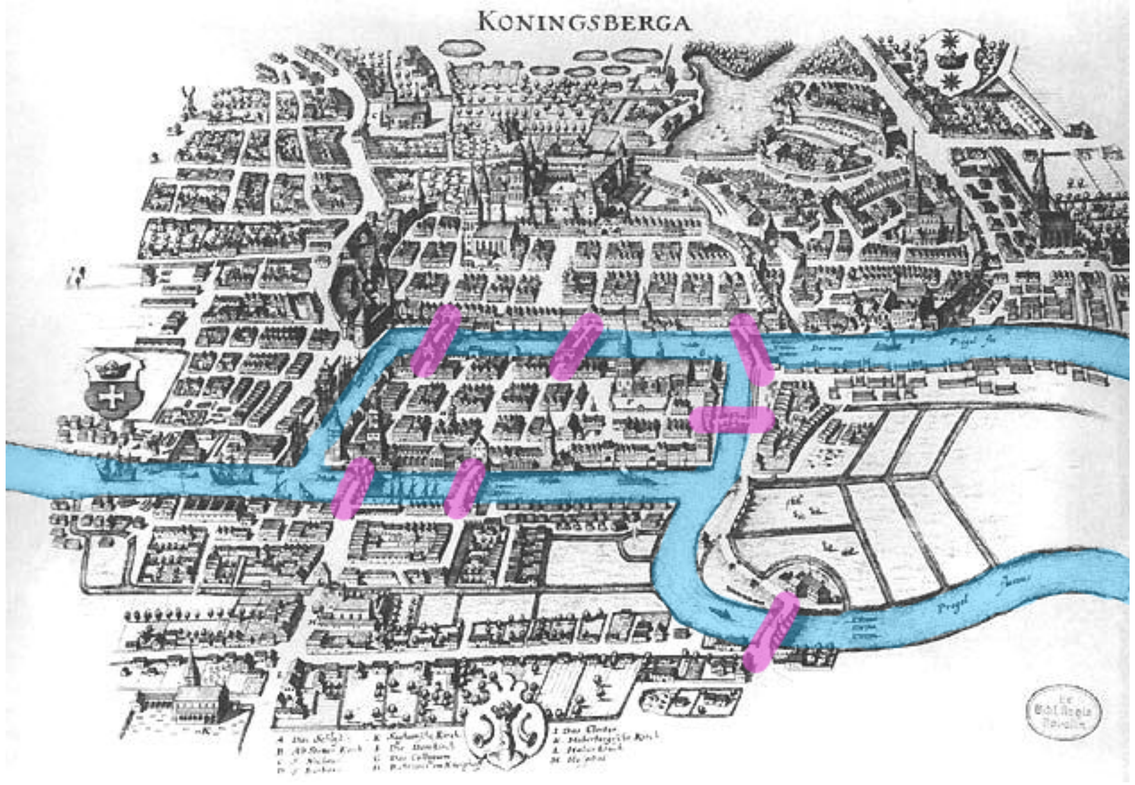
Assume G is connected.

How would a Turing Machine compute this?

How would you describe the input?

Number the vertices $\{1, \cdots, m\}$.

Let the input start with $\mathrm{bin}(m)$.



Given an undirected graph, the existence of an Eulerian cycle is decidable in $O(|V| + |E|)$.

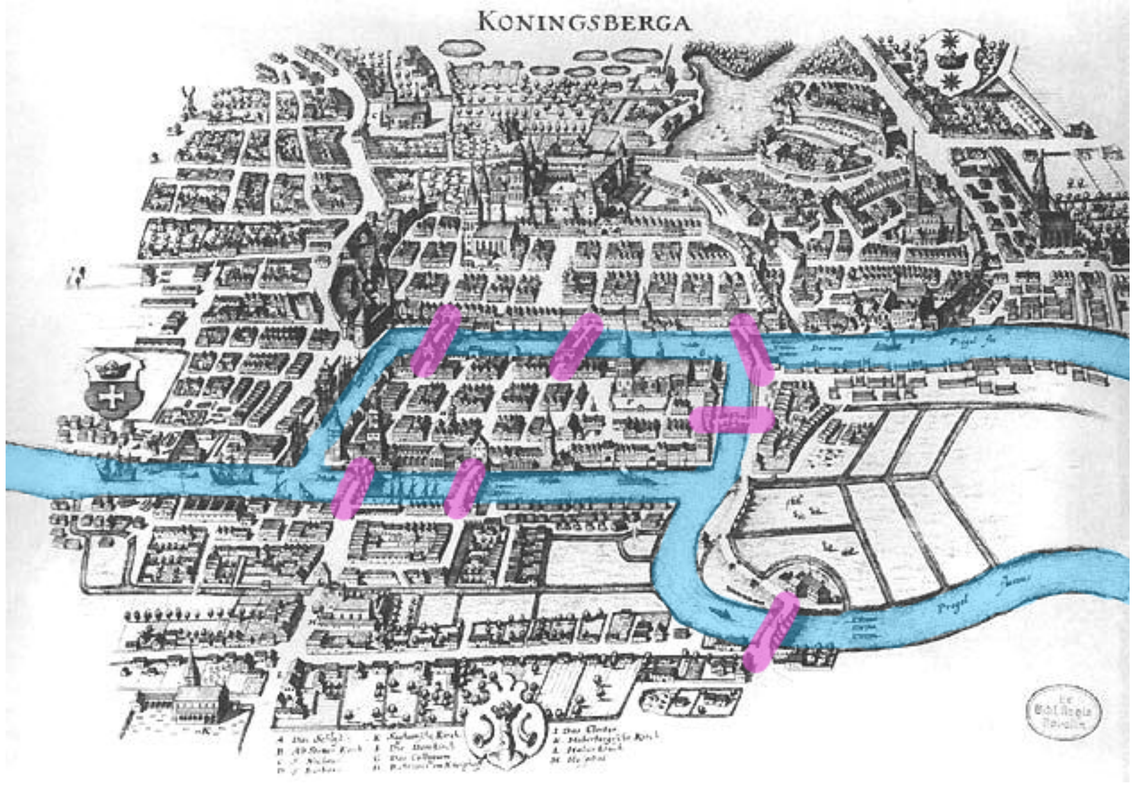
Assume G is connected.

How would a Turing Machine compute this?

How would you describe the input?

Add a separator $\$$.

If there is an edge (i, j) add $\mathrm{bin}(i)\#\mathrm{bin}(j)$.



Given an undirected graph, the existence of an Eulerian cycle is decidable in $O(|V| + |E|)$.

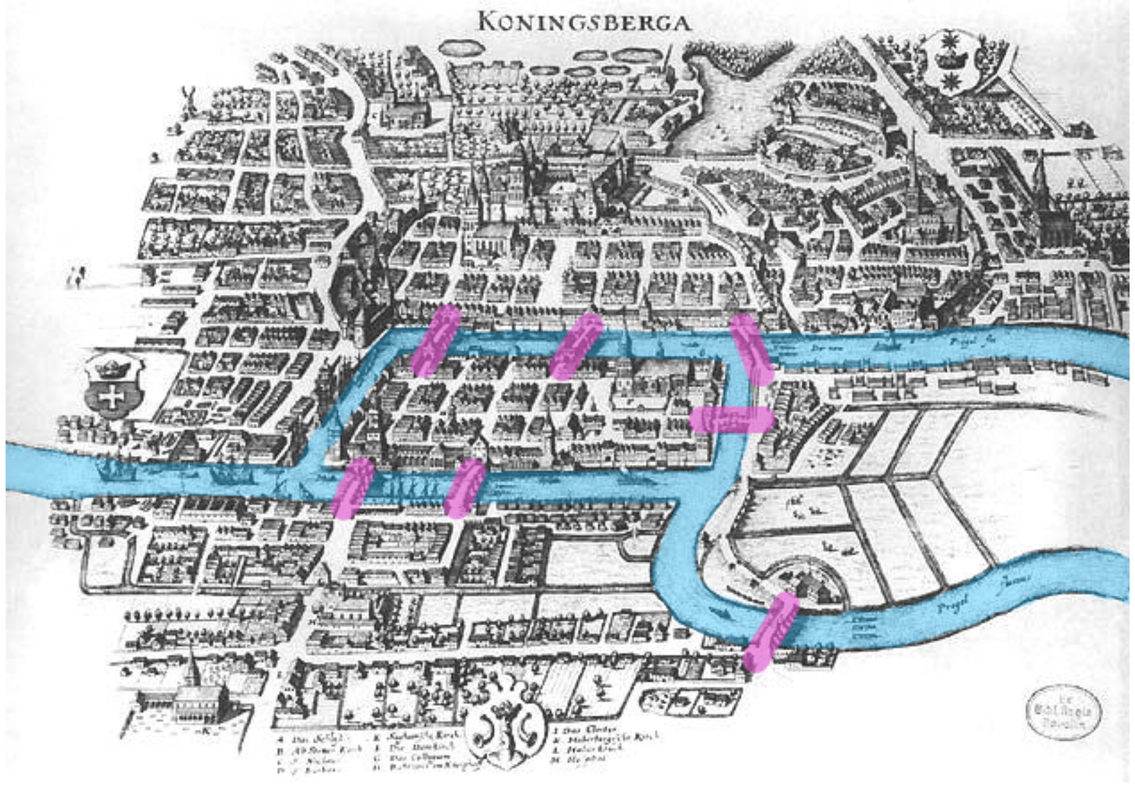
Assume G is connected.

How would a Turing Machine compute this?

How would you describe the input?

Each number is about $k = \log_2(|V|)$.

Description is $k + 1 + (2k + 1)|E|$.



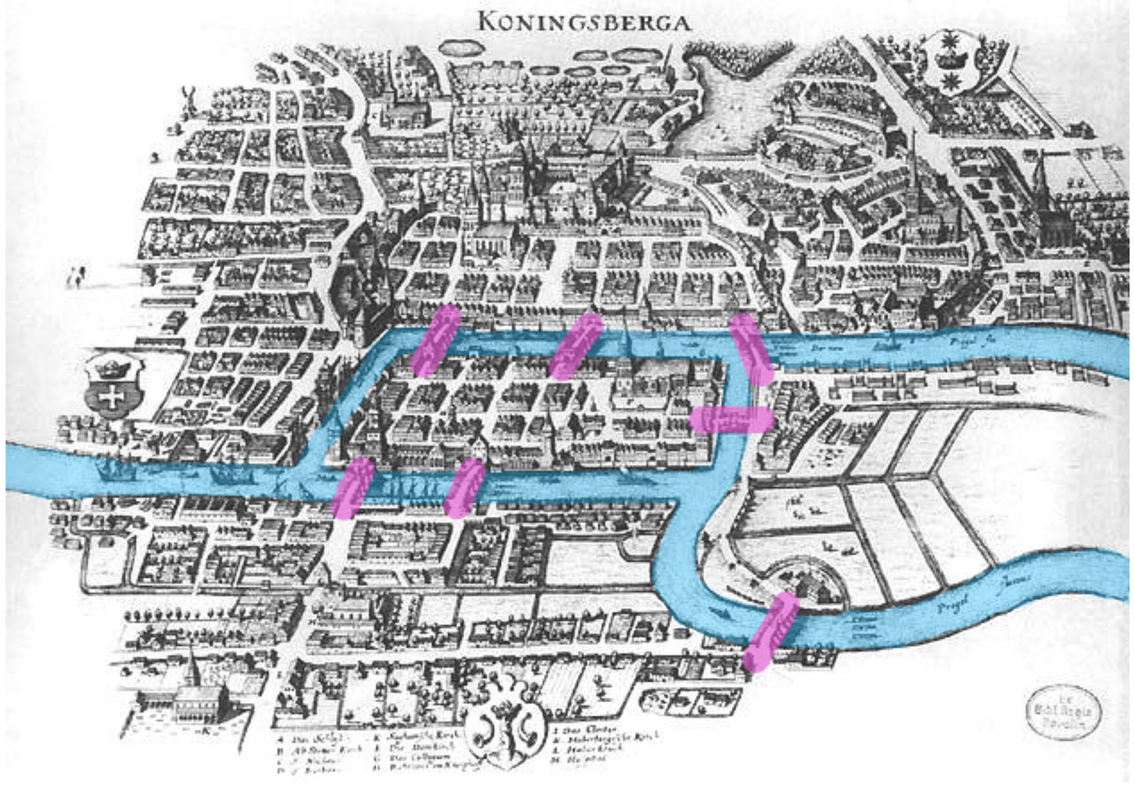
Given an undirected graph, the existence of an Eulerian cycle is decidable in $O(|V| + |E|)$.

Assume G is connected.

How would a Turing Machine compute this?

How would you describe the input?

Description is $O(\ln(|V|) \cdot |E|)$.



Given an undirected graph, the existence of an Eulerian cycle is decidable in $O(|V| + |E|)$.

Assume G is connected.

How would a Turing Machine compute this?

Dictionary has length $O(|V|\ln(|V|))$.

For each edge, we update this dictionary twice. We finally check if each entry is even.

$$O(|E| \cdot |V|\ln(|V|))$$

Why this difference?

DSA teaches you

$$O(|V| + |E|).$$

TOC teaches you

$$O(|E| \cdot |V| \ln(|V|))$$

Why this difference?

DSA teaches you

$$O(|V| + |E|).$$

TOC teaches you

$$O(|E| \cdot |V| \ln(|V|))$$

```
EulerianCycle(G):
    for v in V:
        degree[v] = 0

    for each edge (u, v) in E:
        degree[u]++
        degree[v]++

    for v in V:
        if degree[v] % 2 == 1:
            return NO

    return YES
```

Why this difference?

DSA teaches you

$O(|V| + |E|)$.

TOC teaches you

$O(|E| \cdot |V| \ln(|V|))$

```
EulerianCycle(G):
    for v in V:
        degree[v] = 0

    for each edge (u, v) in E:
        degree[u]++
        degree[v]++

    for v in V:
        if degree[v] % 2 == 1:
            return NO

    return YES
```

$O(1)$?

Why this difference?

DSA teaches you

$$O(|V| + |E|).$$

TOC teaches you

$$O(|E| \cdot |V| \ln(|V|))$$

```
EulerianCycle(G):
    for v in V:
        degree[v] = 0

    for each edge (u, v) in E:
        degree[u]++
        degree[v]++

    for v in V:
        if degree[v] % 2 == 1:
            return NO

    return YES
```

Depends on the model of computation!

IBM 7330
IBM 1401

Aug. 30, 1960 F. C. WILLIAMS 2,951,176

APPARATUS FOR STORING TRAINS OF PULSES

Filed Dec. 10, 1947 3 Sheets-Sheet 1

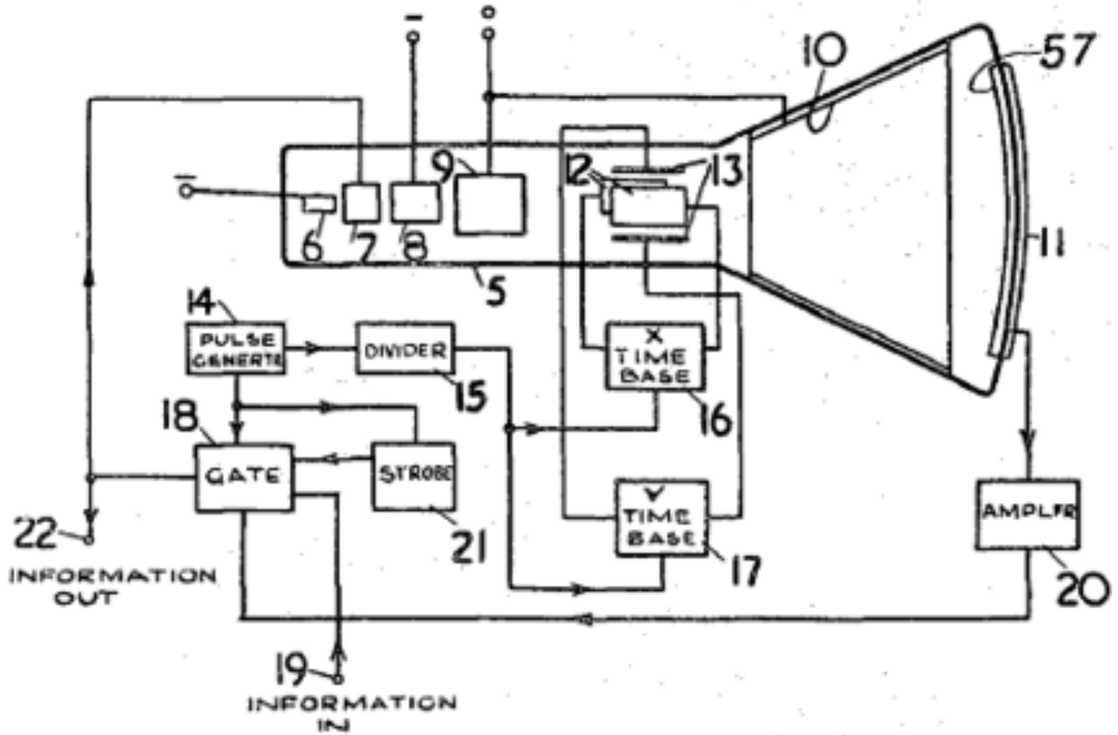
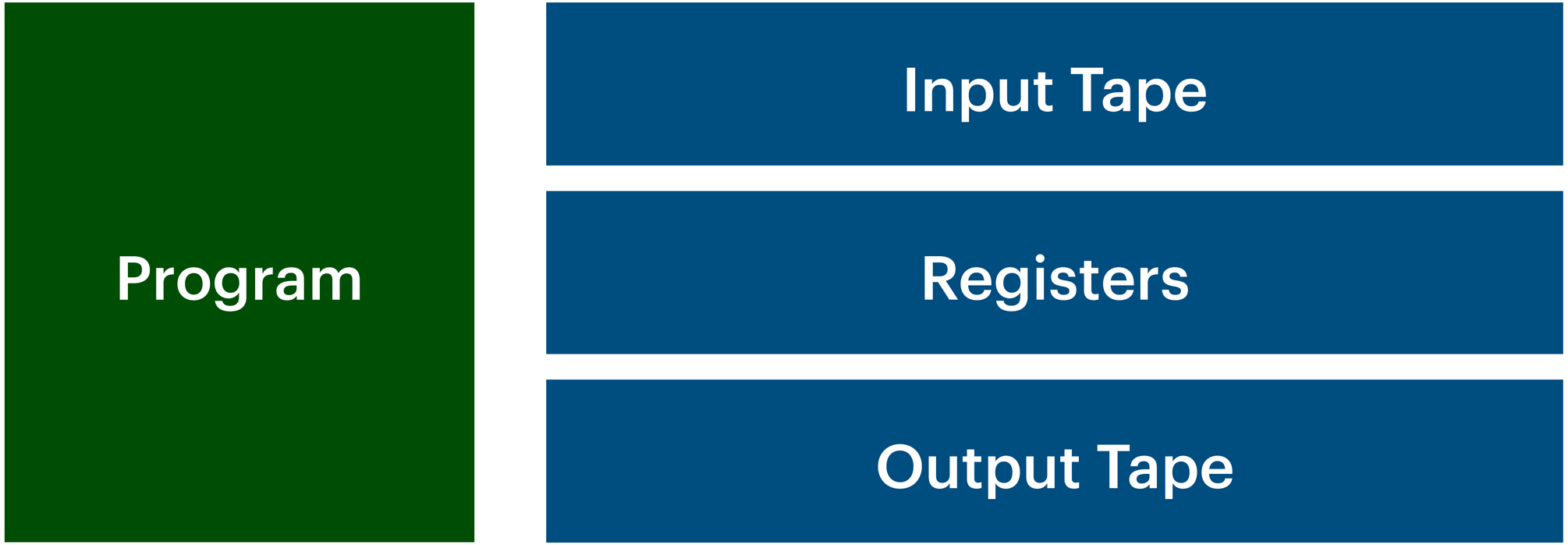


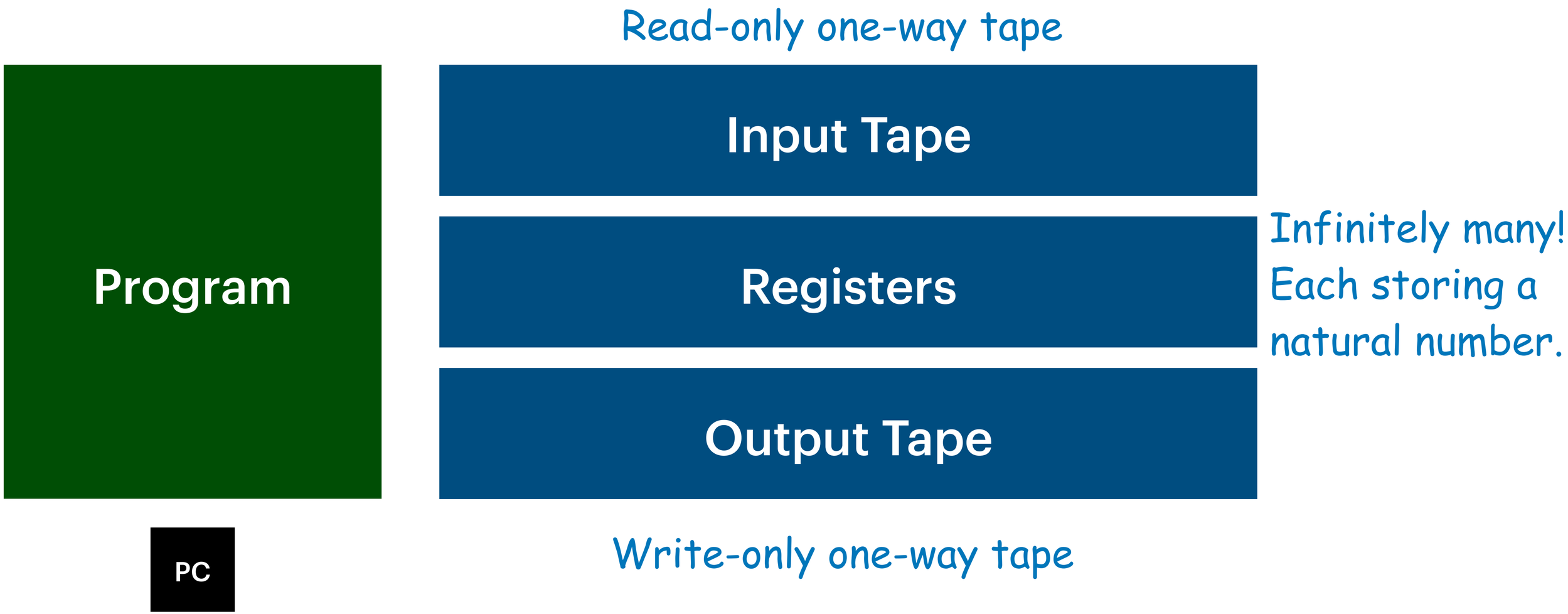
Fig. 1.

RAM Model

RAM Model



RAM Model

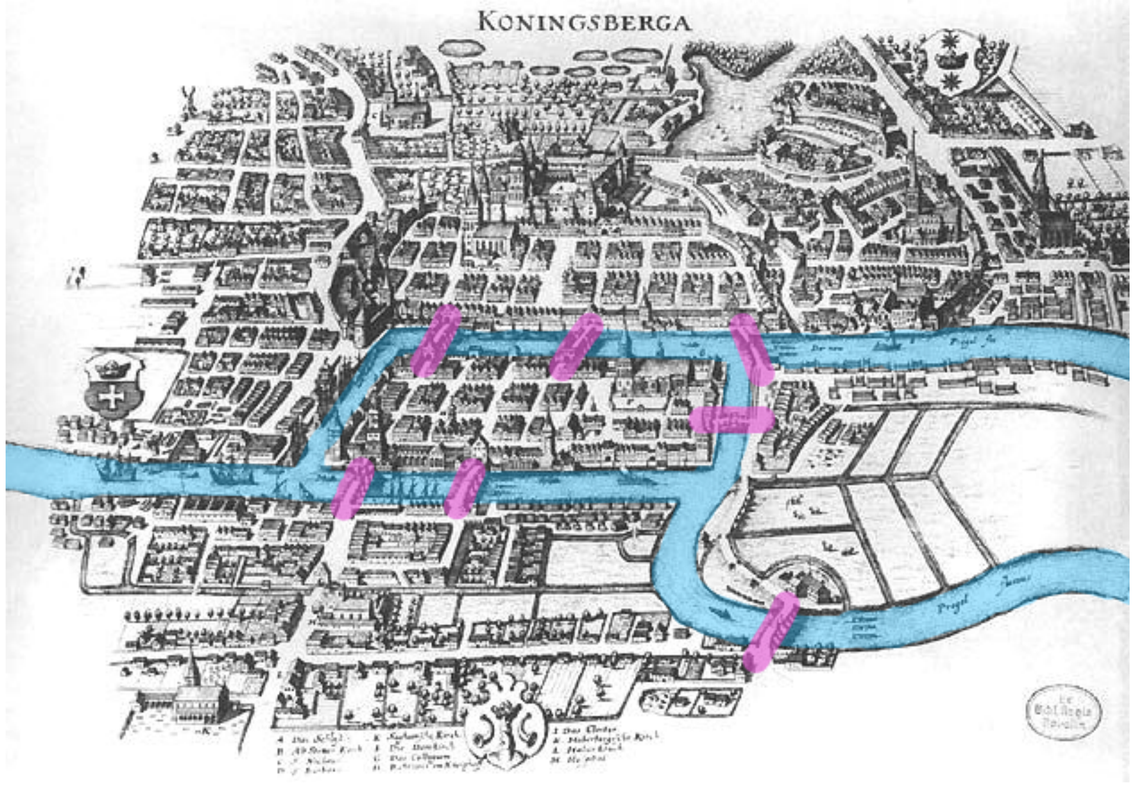

Read-only one-way tape
Program
Input Tape
Registers
Output Tape
Infinitely many!
Each storing a
natural number.
PC
Write-only one-way tape

RAM Model

Machine Instructions (COS):

1. Data Transfers (Load, Store Value, Store Pointer)
2. Arithmetic (Add, Subtract, Multiply, Integer Division)
3. Control Flow (Jumps, Conditional Jump, Halt)
4. Input/Output (Read, Write)

Each instruction costs exactly 1 unit time.



Given an undirected graph, the existence of an Eulerian path is decidable in $O(|V| + |E|)$ in RAM model.

Assume G is connected.

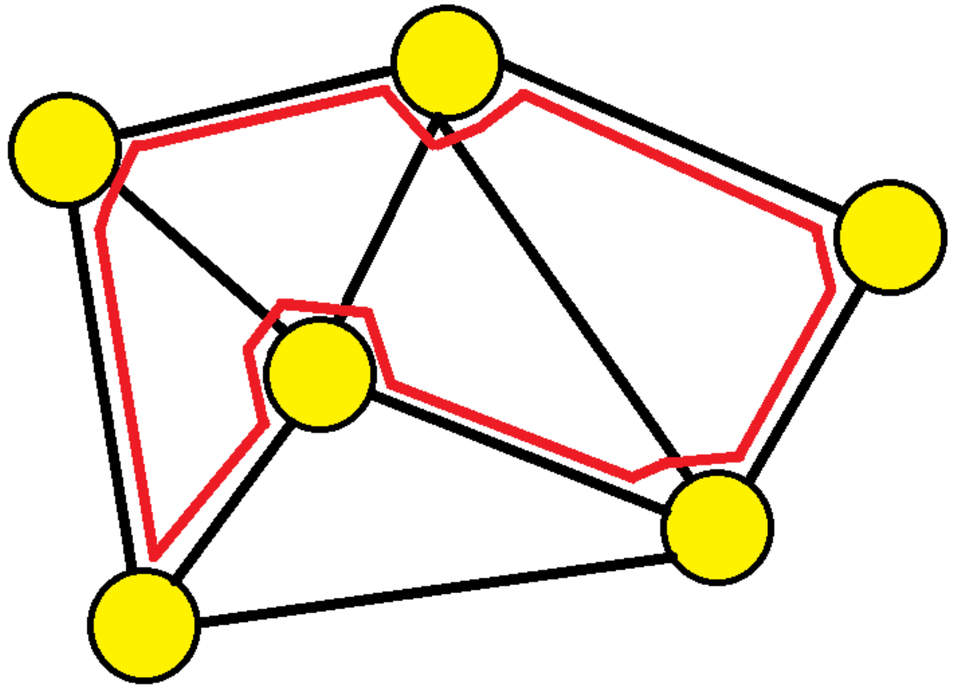
How would a RAM model compute this?

Let the registers store the dictionary.

Parse the input and update the registers.

Finally, parse the registers and check if every value is even.

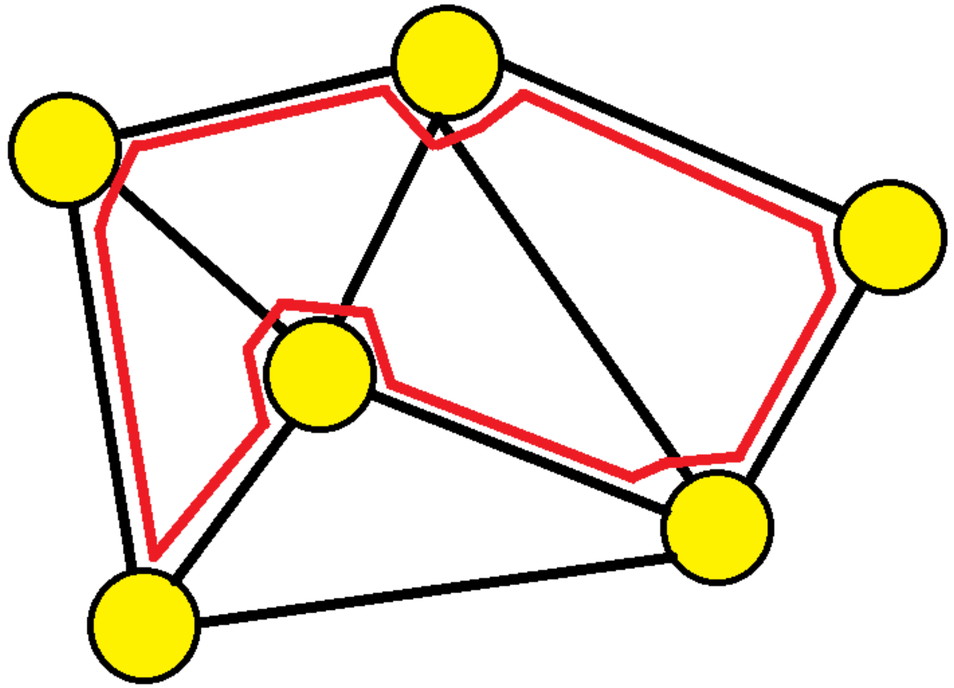
$$O(|V| + |E|)$$



Given an undirected graph, the existence of a Hamiltonian path is decidable in $O(|V|!)$ in RAM model.

Assume G is connected.

How would a RAM model compute this?

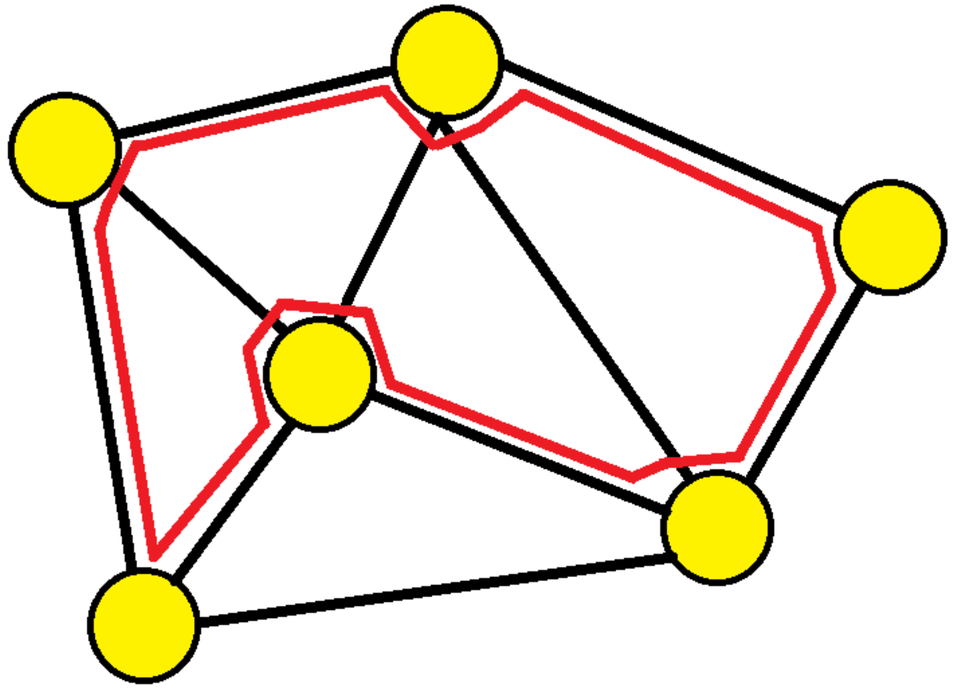


Given an undirected graph, the existence of a Hamiltonian path is decidable in $O(|V|!)$ in RAM model.

Assume G is connected.

How would a RAM model compute this?

How would a deterministic Turing Machine compute this?



Given an undirected graph, the existence of a Hamiltonian path is decidable in $O(|V|!)$ in RAM model.

Assume G is connected.

How would a RAM model compute this?

How would a deterministic Turing Machine compute this?

How would a non-deterministic Turing Machine compute this?

RAM Model

If a TM can solve a problem in $O(T(n))$ time, then a RAM model can solve it in $O(T(n))$.

If a RAM model can solve a problem in $O(T(n))$ time, then a TM can solve it in $O(T(n)^2)$.

Model-Dependent Time Complexity

If a TM can solve a problem in $O(T(n))$ time, then a RAM model can solve it in $O(T(n))$.

If a RAM model can solve a problem in $O(T(n))$ time, then a TM can solve it in $O(T(n)^2)$.

Model-Dependent Time Complexity

If a TM can solve a problem in $O(T(n))$ time, then a RAM model can solve it in $O(T(n))$.

If a RAM model can solve a problem in $O(T(n))$ time, then a TM can solve it in $O(T(n)^2)$.

But what if I only allow my RAM registers to store bits?

Model-Dependent Time Complexity

If a TM can solve a problem in $O(T(n))$ time, then a RAM model can solve it in $O(T(n))$.

If a RAM model can solve a problem in $O(T(n))$ time, then a bit-RAM can solve it in $O(T(n)\log(T(n))^2)$.

But what if I only allow my RAM registers to store bits?

Model-Agnostic Time Complexity

Folk-lore: If a deterministic model of computation can solve a problem in $O(T(n))$, then a deterministic Turing Machine can solve it in $O(\mathrm{poly}(T(n)))$.

Model-Agnostic Time Complexity

Folk-lore: If a deterministic model of computation can solve a problem in $O(T(n))$, then a deterministic Turing Machine can solve it in $O(\mathrm{poly}(T(n)))$.

Hence we study the class of deterministic polynomial time instead of linear time, quadratic time, etc.

Model-Agnostic Time Complexity

Let $\mathbf{P}$ denote the collection of all recursive languages for which membership can be decided in *polynomial time* by a deterministic Turing Machine.

Model-Agnostic Time Complexity

Let $\mathbf{P}$ denote the collection of all recursive languages for which membership can be decided in *polynomial time* by a deterministic Turing Machine.

Prove that $\mathbf{P}$ is closed under the following:

Boolean Combination, Concatenation, Kleene star, Homomorphisms

Model-Agnostic Time Complexity

Let $\mathbf{NP}$ denote the collection of all recursive languages for which membership can be decided in *polynomial time* by a non-deterministic Turing Machine.

Prove that $\mathbf{NP}$ is closed under the following:

Boolean Combination, Concatenation, Kleene star, Homomorphisms

Model-Agnostic Time Complexity

Let $\mathbf{NP}$ denote the collection of all recursive languages for which membership can be decided in *polynomial time* by a non-deterministic Turing Machine.

Prove that $\mathbf{NP}$ is closed under the following:

Boolean Combination, Concatenation, Kleene star, Homomorphisms

Additionally $\mathbf{P} \subseteq \mathbf{NP}$.

Turing Reductions

Suppose we have $L_1 \subseteq \Sigma^*$ and $L_2 \subseteq \Gamma^*$,
and a function $f : \Sigma^* \to \Gamma^*$ such that:

1. f is computable
2. $x \in L_1 \iff f(x) \in L_2$

Then we say $L_1 \leq_T L_2$.

Turing Reductions

Suppose we have $L_1 \subseteq \Sigma^*$ and $L_2 \subseteq \Gamma^*$,
and a function $f : \Sigma^* \to \Gamma^*$ such that:

1. f is computable
2. $x \in L_1 \iff f(x) \in L_2$

Then we say $L_1 \leq_T L_2$.

If L_2 is recursive then so is L_1.

Karp Reductions

Suppose we have $L_1 \subseteq \Sigma^*$ and $L_2 \subseteq \Gamma^*$,
and a function $f : \Sigma^* \to \Gamma^*$ such that:

1. f can be computed in polynomial time.
2. $x \in L_1 \iff f(x) \in L_2$

Then we say $L_1 \leq_P L_2$.

Karp Reductions

Suppose we have $L_1 \subseteq \Sigma^*$ and $L_2 \subseteq \Gamma^*$,
and a function $f : \Sigma^* \to \Gamma^*$ such that:

1. f can be computed in polynomial time.
2. $x \in L_1 \iff f(x) \in L_2$

Then we say $L_1 \leq_P L_2$.

Clearly: $L_2 \in \mathbf{P} \implies L_1 \in \mathbf{P}$

Karp Reductions

Suppose we have $L_1 \subseteq \Sigma^*$ and $L_2 \subseteq \Gamma^*$,
and a function $f : \Sigma^* \to \Gamma^*$ such that:

1. f can be computed in polynomial time.
2. $x \in L_1 \iff f(x) \in L_2$

Then we say $L_1 \leq_P L_2$.

Clearly: $L_2 \in \mathbf{NP} \implies L_1 \in \mathbf{NP}$

Karp Reductions

Therefore $\mathbf{P}$ and $\mathbf{NP}$ are both downward closed under $\leq_P$.

Karp Reductions

Therefore $\mathbf{P}$ and $\mathbf{NP}$ are both downward closed under $\leq_P$.

Suppose $\mathrm{ALL}_{\mathrm{TM}} \in \mathbf{NP}$.

What can you say about $\mathrm{ALL}_{\mathrm{DFA}}$?